DUEL MASTERS TCG ENGINE

Strategy Analysis, Feasibility Review & Full Project Codebase

GODOT 4.X (C# MONO) .NET 9 BACKEND PYTHON INGESTION MASTER DUEL AESTHETICS

1. Strategy Analysis & Feasibility Review

Recreating a modern digital version of **Duel Masters** using **Godot 4.x (C# Edition)** as the rendering client and **.NET 9** as the authoritative server is an exceptionally strong architecture choice. Below is the full strategy breakdown:

1. Shared C# Domain Model

Because both Godot 4 (Mono C#) and .NET 9 run on modern .NET runtimes, you can share a single core game domain assembly (`DuelMasters.Domain`) between client and server. This avoids duplicating rules logic or state machine code across multiple languages.

2. Master Duel Visual Fidelity in Godot 4

Godot 4's rendering pipeline (2.5D viewport camera with Y-sort, custom fragment shaders for card dissolution, and GPU particles for energy bursts) easily reaches the cinematic feel of titles like *Yu-Gi-Oh! Master Duel* and *Power of Chaos*.

3. Decoupled & Protocol-Agnostic Backend

By establishing a pure JSON event schema (WebSocket/SignalR), your backend logic runs on .NET 9 today and can easily be ported to **NestJS (Socket.io)** or **Spring Boot (STOMP)** in the future without changing any visual UI code in Godot.

4. Data Ingestion Solution for Raw Card Images

Because the card image collection consists of raw `.png` or `.jpeg` assets without structured text data, an automated Python data ingestion script acts as the single source of truth, converting visual assets into a unified `cards.json` database.

2. Complete Project Codebase Specification

2.1 Data Ingestion Pipeline (`ingest_cards.py`)

```
 import os
import json
import glob
from pathlib import Path

# Directory configuration
IMAGE_DIR = "./assets/cards_raw"
OUTPUT_JSON = "./data/cards.json"

DEFAULT_CARD_TEMPLATES = {
    "dm_01_001": {
        "name": "Rothus, the Traveler",
        "civilization": "Fire",
        "cardType": "Creature",
        "manaCost": 4,
        "power": 4000,
        "race": "Armor Dragon",
        "keywords": ["POWER_ATTACKER_2000"],
        "scriptEffectId": "EFFECT_ROTHUS_DESTROY_SELF"
    },
    "dm_01_002": {
        "name": "Aqua Vehicle",
        "civilization": "Water",
        "cardType": "Creature",
        "manaCost": 2,
        "power": 1000,
        "race": "Liquid People",
        "keywords": [],
        "scriptEffectId": "VANILLA"
    },
    "dm_01_003": {
        "name": "Holy Awe",
        "civilization": "Light",
        "cardType": "Spell",
        "manaCost": 6,
        "power": 0,
        "race": "Spell",
        "keywords": ["SHIELD_TRIGGER"],
        "scriptEffectId": "EFFECT_TAP_ALL_ENEMY_CREATURES"
    }
}

def generate_card_database():
    card_database = []
    image_files = glob.glob(os.path.join(IMAGE_DIR, "*.[pj][np][eg]*"))

    print(f"[Pipeline] Found {len(image_files)} card images in {IMAGE_DIR}")

    for filepath in sorted(image_files):
        filename = Path(filepath).stem
        ext = Path(filepath).suffix
        relative_path = f"res://assets/cards/{filename}{ext}"

        # Check if template metadata exists, else assign default structured payload
        meta = DEFAULT_CARD_TEMPLATES.get(filename, {
            "name": filename.replace("_", " ").title(),
            "civilization": "Fire",
            "cardType": "Creature",
            "manaCost": 3,
            "power": 2000,
            "race": "Dragonoid",
            "keywords": [],
            "scriptEffectId": "VANILLA"
        })

        card_entry = {
            "id": filename,
            "name": meta["name"],
            "civilization": meta["civilization"],
```

```
            "cardType": meta["cardType"],
            "manaCost": meta["manaCost"],
            "manaNumber": 1,
            "power": meta["power"],
            "race": meta["race"],
            "imagePath": relative_path,
            "keywords": meta["keywords"],
            "scriptEffectId": meta["scriptEffectId"]
        }
        card_database.append(card_entry)

    os.makedirs(os.path.dirname(OUTPUT_JSON), exist_ok=True)
    with open(OUTPUT_JSON, "w", encoding="utf-8") as f:
        json.dump(card_database, f, indent=2)

    print(f"[Pipeline] Successfully exported {len(card_database)} cards to {OUTPUT_JSON}")

if __name__ == "__main__":
    generate_card_database()
```

2.2 Master Card Database Schema (`cards.json`)

```
[
  {
    "id": "dm_01_001",
    "name": "Rothus, the Traveler",
    "civilization": "Fire",
    "cardType": "Creature",
    "manaCost": 4,
    "manaNumber": 1,
    "power": 4000,
    "race": "Armor Dragon",
    "imagePath": "res://assets/cards/dm_01_001.png",
    "keywords": [
      "POWER_ATTACKER_2000"
    ],
    "scriptEffectId": "EFFECT_ROTHUS_DESTROY_SELF"
  },
  {
    "id": "dm_01_002",
    "name": "Aqua Vehicle",
    "civilization": "Water",
    "cardType": "Creature",
    "manaCost": 2,
    "manaNumber": 1,
    "power": 1000,
    "race": "Liquid People",
    "imagePath": "res://assets/cards/dm_01_002.png",
    "keywords": [],
    "scriptEffectId": "VANILLA"
  },
  {
    "id": "dm_01_003",
    "name": "Holy Awe",
    "civilization": "Light",
    "cardType": "Spell",
    "manaCost": 6,
    "manaNumber": 1,
    "power": 0,
    "race": "Spell",
    "imagePath": "res://assets/cards/dm_01_003.png",
    "keywords": [
      "SHIELD_TRIGGER"
    ],
    "scriptEffectId": "EFFECT_TAP_ALL_ENEMY_CREATURES"
  }
]
```

2.3 Godot 4 Card View Node (`CardNode.cs`)

```
using Godot;
using System;

public partial class CardNode : Area2D
{
    [Signal] public delegate void CardClickedEventHandler(string cardId);
    [Signal] public delegate void CardHoveredEventHandler(string cardId, bool isHovered);

    [Export] public Sprite2D CardArtwork { get; set; }
    [Export] public Label PowerLabel { get; set; }
    [Export] public Label ManaCostLabel { get; set; }
    [Export] public Control OutlineHighlight { get; set; }

    public string CardId { get; private set; }
    public bool IsTapped { get; private set; } = false;
    private Vector2 _targetPosition;
    private float _targetRotation = 0f;

    public override void _Ready()
    {
        MouseEntered += OnMouseEntered;
        MouseExited += OnMouseExited;
    }

    public void SetupCard(string id, string artworkPath, int power, int manaCost)
    {
        CardId = id;
        PowerLabel.Text = power > 0 ? power.ToString() : "";
        ManaCostLabel.Text = manaCost.ToString();

        var texture = GD.Load<Texture2D>(artworkPath);
        if (texture != null)
        {
            CardArtwork.Texture = texture;
        }
    }

    public override void _Process(double delta)
    {
        // Smooth interpolation for position and rotation lerping
        Position = Position.Lerp(_targetPosition, (float)delta * 12.0f);
        Rotation = Mathf.LerpAngle(Rotation, _targetRotation, (float)delta * 12.0f);
    }

    public void SetTappedState(bool tapped)
    {
        IsTapped = tapped;
        _targetRotation = tapped ? Mathf.DegToRad(90.0f) : 0.0f;
    }

    public void MoveTo(Vector2 newPos)
    {
        _targetPosition = newPos;
    }

    private void OnMouseEntered()
    {
        EmitSignal(SignalName.CardHovered, CardId, true);
        Scale = new Vector2(1.15f, 1.15f);
    }

    private void OnMouseExited()
    {
        EmitSignal(SignalName.CardHovered, CardId, false);
        Scale = Vector2.One;
    }

    public override void _InputEvent(Godot.Object viewport, InputEvent @event, long shapeIdx)
    {
```

```
        if (@event is InputEventMouseButton mouseBtn && mouseBtn.Pressed && mouseBtn.ButtonIndex ==
MouseButton.Left)
        {
            EmitSignal(SignalName.CardClicked, CardId);
        }
    }
}
```

2.4 Godot 4 Board Controller (`BoardController.cs`)

```
 using Godot;
using System;
using System.Collections.Generic;

public partial class BoardController : Node2D
{
    [Export] public PackedScene CardNodePrefab { get; set; }
    [Export] public Node2D BattleZoneContainer { get; set; }
    [Export] public Node2D ManaZoneContainer { get; set; }
    [Export] public Control HandContainer { get; set; }

    private Dictionary<string, CardNode> _activeCards = new();

    public override void _Ready()
    {
        // Connect network state listener
        NetworkManager.Instance.OnGameStateUpdated += UpdateBoardState;
    }

    public void UpdateBoardState(GameStateDto state)
    {
        // Synchronize Hand Cards
        foreach (var cardDto in state.ActivePlayerHand)
        {
            if (!_activeCards.ContainsKey(cardDto.InstanceId))
            {
                var newCard = CardNodePrefab.Instantiate<CardNode>();
                HandContainer.AddChild(newCard);
                newCard.SetupCard(cardDto.CardId, cardDto.ImagePath, cardDto.Power, cardDto.ManaCost);
                newCard.CardClicked += (id) => OnHandCardSelected(cardDto.InstanceId);
                _activeCards[cardDto.InstanceId] = newCard;
            }
        }

        // Synchronize Mana Zone Tapped States & Layout
        for (int i = 0; i < state.ActivePlayerMana.Count; i++)
        {
            var manaDto = state.ActivePlayerMana[i];
            if (_activeCards.TryGetValue(manaDto.InstanceId, out var cardNode))
            {
                cardNode.Reparent(ManaZoneContainer);
                cardNode.MoveTo(new Vector2(i * 60.0f, 0));
                cardNode.SetTappedState(manaDto.IsTapped);
            }
        }
    }

    private void OnHandCardSelected(string instanceId)
    {
        GD.Print($"[Client] Card clicked in hand: {instanceId}");
        NetworkManager.Instance.SendPlayCardRequest(instanceId);
    }
}
```

2.5 Domain Game Engine State (`DuelGameState.cs`)

```
namespace DuelMasters.Domain;

public enum TurnPhase
{
    Untap,
    Draw,
    Mana,
    Main,
    Attack,
    End
}

public class CardInstance
{
    public string InstanceId { get; set; } = Guid.NewGuid().ToString();
    public string CardId { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Civilization { get; set; } = string.Empty;
    public string CardType { get; set; } = string.Empty; // Creature, Spell, Evolution
    public int ManaCost { get; set; }
    public int Power { get; set; }
    public bool IsTapped { get; set; } = false;
    public bool HasSummoningSickness { get; set; } = true;
    public List<string> Keywords { get; set; } = new();
}

public class PlayerState
{
    public string PlayerId { get; set; } = string.Empty;
    public List<CardInstance> Deck { get; set; } = new();
    public List<CardInstance> Hand { get; set; } = new();
    public List<CardInstance> ManaZone { get; set; } = new();
    public List<CardInstance> BattleZone { get; set; } = new();
    public List<CardInstance> Shields { get; set; } = new();
    public List<CardInstance> Graveyard { get; set; } = new();
}

public class DuelGameState
{
    public string MatchId { get; set; } = Guid.NewGuid().ToString();
    public PlayerState Player1 { get; set; } = new();
    public PlayerState Player2 { get; set; } = new();
    public string ActivePlayerId { get; set; } = string.Empty;
    public TurnPhase CurrentPhase { get; set; } = TurnPhase.Untap;
    public int TurnCount { get; set; } = 1;
    public bool IsGameOver { get; set; } = false;
    public string WinnerPlayerId { get; set; } = string.Empty;

    public bool PlayCardToMana(string playerId, string instanceId)
    {
        if (ActivePlayerId != playerId || CurrentPhase != TurnPhase.Mana) return false;

        var player = GetPlayer(playerId);
        var card = player.Hand.FirstOrDefault(c => c.InstanceId == instanceId);
        if (card == null) return false;

        player.Hand.Remove(card);
        player.ManaZone.Add(card);
        CurrentPhase = TurnPhase.Main; // Transition to Main Phase
        return true;
    }

    public bool DeclareAttack(string attackerId, string targetId, out string resultSummary)
    {
        resultSummary = "";
        var attackerPlayer = GetPlayer(ActivePlayerId);
        var defenderPlayer = GetOpponent(ActivePlayerId);

        var attackerCard = attackerPlayer.BattleZone.FirstOrDefault(c => c.InstanceId == attackerId);
```

```
        if (attackerCard == null || attackerCard.IsTapped || attackerCard.HasSummoningSickness)
            return false;

        attackerCard.IsTapped = true; // Tap attacker

        // Shield Break Resolution
        var targetShield = defenderPlayer.Shields.FirstOrDefault(s => s.InstanceId == targetId);
        if (targetShield != null)
        {
            defenderPlayer.Shields.Remove(targetShield);
            defenderPlayer.Hand.Add(targetShield); // Broken shield goes to defender hand
            resultSummary = $"SHIELD_BROKEN:{targetShield.InstanceId}";

            if (defenderPlayer.Shields.Count == 0 && attackerPlayer.BattleZone.Any(c => c.InstanceId ==
attackerId))
            {
                // Win Condition Check
                IsGameOver = true;
                WinnerPlayerId = ActivePlayerId;
                resultSummary = "GAME_OVER_VICTORY";
            }
            return true;
        }

        return false;
    }

    public PlayerState GetPlayer(string id) => Player1.PlayerId == id ? Player1 : Player2;
    public PlayerState GetOpponent(string id) => Player1.PlayerId == id ? Player2 : Player1;
}
```

2.6 Authoritative .NET 9 Server Hub (`DuelHub.cs`)

```
using Microsoft.AspNetCore.SignalR;
using DuelMasters.Domain;

namespace DuelMasters.Infrastructure;

public interface IDuelClientContract
{
    Task ReceiveGameState(DuelGameState state);
    Task ReceiveActionError(string errorMessage);
    Task TriggerShieldBreakVFX(string shieldInstanceId, bool isShieldTrigger);
    Task AnnounceWinner(string winnerPlayerId);
}

public class DuelHub : Hub<IDuelClientContract>
{
    private static readonly Dictionary<string, DuelGameState> ActiveMatches = new();

    public async Task PlayManaCard(string matchId, string cardInstanceId)
    {
        if (!ActiveMatches.TryGetValue(matchId, out var match))
        {
            await Clients.Caller.ReceiveActionError("Match session not found.");
            return;
        }

        var playerId = Context.ConnectionId;
        bool success = match.PlayCardToMana(playerId, cardInstanceId);

        if (success)
        {
            await Clients.Group(matchId).ReceiveGameState(match);
        }
        else
        {
            await Clients.Caller.ReceiveActionError("Invalid mana play action.");
        }
    }

    public async Task ExecuteAttack(string matchId, string attackerInstanceId, string targetInstanceId)
    {
        if (!ActiveMatches.TryGetValue(matchId, out var match)) return;

        bool success = match.DeclareAttack(attackerInstanceId, targetInstanceId, out string summary);
        if (success)
        {
            if (summary.StartsWith("SHIELD_BROKEN"))
            {
                var shieldId = summary.Split(':')[1];
                await Clients.Group(matchId).TriggerShieldBreakVFX(shieldId, false);
            }

            await Clients.Group(matchId).ReceiveGameState(match);

            if (match.IsGameOver)
            {
                await Clients.Group(matchId).AnnounceWinner(match.WinnerPlayerId);
            }
        }
    }
}
```

3. Step-by-Step Development Execution Plan

1 Phase 1: Asset Pipeline Execution

Place raw `.png` / `.jpeg` card files inside `./assets/cards_raw/` and run `python ingest_cards.py` to generate the initial structured `cards.json` database.

2 Phase 2: Offline C# Domain Engine

Implement the `DuelMasters.Domain` C# class library with zero engine dependencies and execute xUnit tests for turn phases, mana play, creature combat, and shield break handling.

3 Phase 3: Godot 4 Client Visual Arena

Create `CardNode.tscn` with card artwork loader, hover scaling, and 90-degree tap rotation lerp. Set up the 2.5D board camera and run a local hotseat prototype.

4 Phase 4: .NET 9 Authoritative Server Integration

Build the ASP.NET Core 9 Web API project containing `DuelHub.cs` and connect Godot C# client via `Microsoft.AspNetCore.SignalR.Client`.

5 Phase 5: Visual Effects & Polish

Implement Godot fragment shaders for shield shattering disintegrations, screen camera shake, and civilization-themed energy particle beams (Fire flames, Water sapphire waves, Light laser rays).